Evolutionary Test Generation with Diversity-Aware Mutation Fitness

CS453 Team 6

Background: EvoSuite

https://www.evosuite.org/

- What is Evosuite?
  - Open-source tool for SBST
  - Automatically generates JUnit test suites

- How EvoSuite Generates Tests
  - Utilizes an evolutionary algorithm
  - Uses various factors to evaluate **fitness function**
    - Branch distance
    - Code coverage
    - **Mutation score**
    - Test suite properties
    - etc.

Motivation

- Mutation score is based on ‘total killed mutants’.
- However, there are many operator types.

- Let M_1, M_2, M_3 be counts of killed mutants grouped by operator type.
  - For example:
  - M_1 - arithmetic mutations.
  - M_2 - comparison mutations.
  - M_3 - condition negation.

Motivation

- Consider two test suites with following mutation score:

Test Suite	Killed mutants	Mutation Score
A	M_1=100, M_2=10, M_3=10	Same
B	M_1=40, M_2=40, M_3=40	Same

- Which one is better?

Mutation Score as a Fitness Function?

- Some operator types are easier to kill early than others
- Mutation score alone can favor early, easy wins
- Naïve mutation score can create a “blind-spot”
  - The GA may over-invest in easy types and under-explore harder ones

Test Suite	Killed mutants	Mutation Score
A	M_1=100, M_2=10, M_3=10	Same
B	M_1=40, M_2=40, M_3=40	Same

Research Hypothesis

Coupling operator-type diversity into fitness guides early selection toward broader exploration — improving fault-model coverage without sacrificing mutation score.

Approach: Diversity Metric

- Use Shannon entropy over killed-mutant distribution

Diversity metric

Let c_i be killed mutants of operator type i, $p_i = c_i \,/\, \sum_j c_j$, and n the number of operator types in the class:

$$v = -\sum_{i=1}^{n} p_i \log p_i, \qquad \hat{v} = \frac{v}{\log n} \in [0,1], \qquad \delta = 1 - \hat{v}$$

(from project's README.md)

- Nine operator types are tracked

Approach: Coupling Diversity into Fitness

- Choose various candidates
- All having our custom parameter k

Fitness function

EvoSuite minimises fitness (0 = all mutants killed):

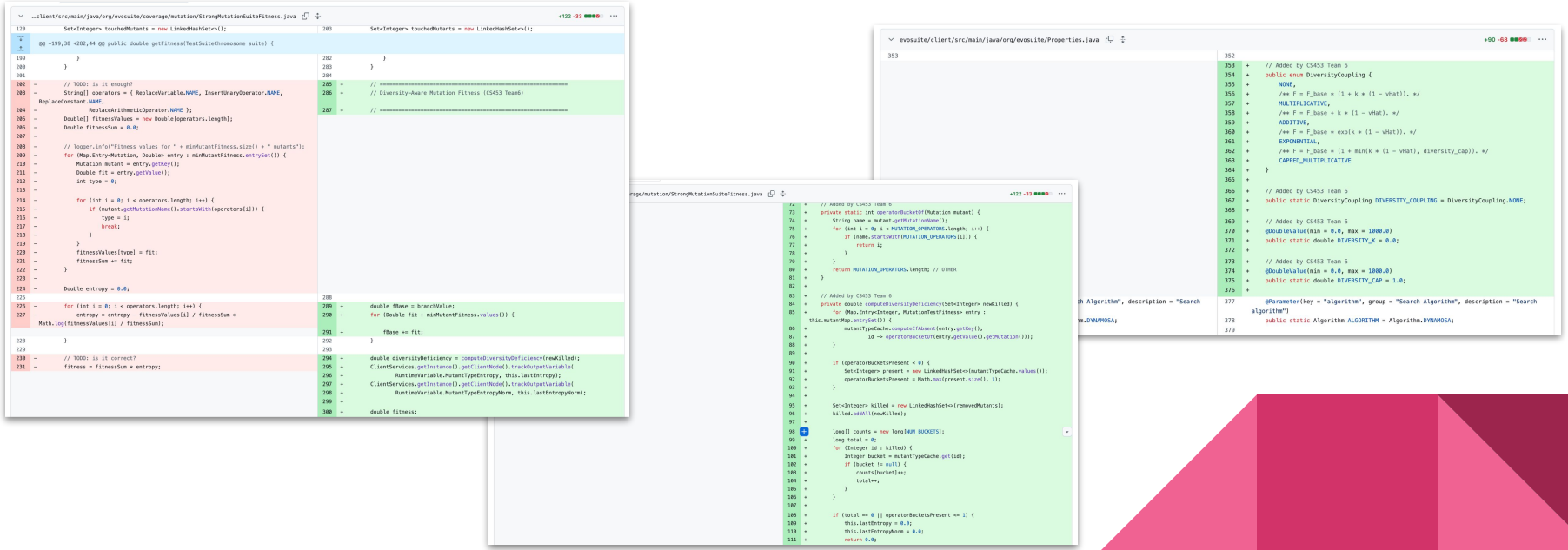
$$F_{\text{base}} = f_{\text{branch}} + \sum_{m \in \text{alive}} d_{\min}(m)$$

Mode	Formula
`NONE`	F_{base}
`MULTIPLICATIVE`	$F_{\text{base}} \cdot (1 + k\delta)$
`ADDITIVE`	$F_{\text{base}} + k\delta$
`EXPONENTIAL`	$F_{\text{base}} \cdot e^{k\delta}$

(from project's README.md)

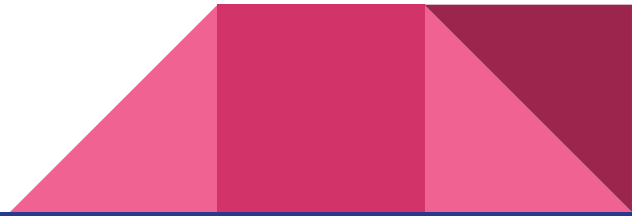
Implementation

- Cloned latest Evosuite from GitHub
- Manually injected our fitness score into Evosuite source code



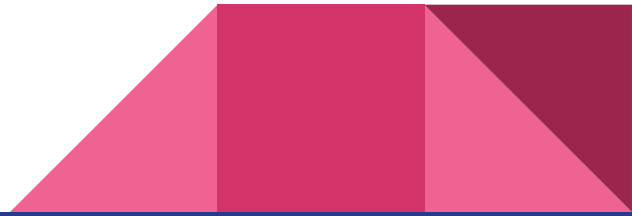
Experiment Strategy

- Choose various open-source library to test
- Run with each candidate fitness function, changing parameter k
- See final result to select best-performance fitness



Experiment: Benchmark Selection

- Diverse Type of Computational domains
- Dense nesting of Conditionals, Loops, & Exception–Good for Branch Reachability
- Pure, Deterministic Math Functions–Good for Infection & Propagation

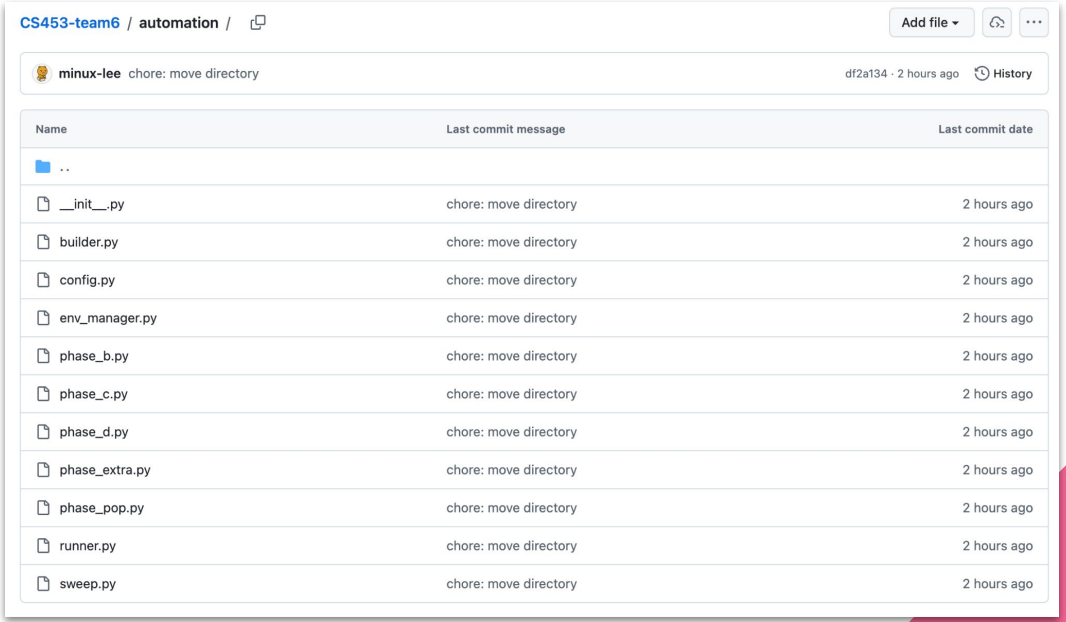


Experiment: Benchmark Selection

Domain	Class	Library	Total Goals
Numerical	ArithmeticUtils	commons-math3	992
	Fraction	commons-math3	709
	Precision	commons-math3	650
	Complex	commons-math3	917
	IntMath	guava	605
	IEEE754rUtils	commons-lang3	176
	NumberUtils	commons-lang3	562
String	CharUtils	commons-lang3	405
	WordUtils	commons-text	827
	Strings	guava	442
	LevenshteinDist.	commons-text	224
Logic	BooleanUtils	commons-lang3	811
Encoding	Hex	commons-codec	379
	Soundex	commons-codec	241

Experiment: Automation

- Had to run through thousands of runs with each taking about a minute
- Wrote automation script in python



CS453-team6 / automation /

Add file

minux-lee chore: move directory — df2a134 · 2 hours ago — History

Name	Last commit message	Last commit date
..		
__init__.py	chore: move directory	2 hours ago
builder.py	chore: move directory	2 hours ago
config.py	chore: move directory	2 hours ago
env_manager.py	chore: move directory	2 hours ago
phase_b.py	chore: move directory	2 hours ago
phase_c.py	chore: move directory	2 hours ago
phase_d.py	chore: move directory	2 hours ago
phase_extra.py	chore: move directory	2 hours ago
phase_pop.py	chore: move directory	2 hours ago
runner.py	chore: move directory	2 hours ago
sweep.py	chore: move directory	2 hours ago

Experiment: Automation

- Challenge: Computational Limitations
  - 14 libraries, 4 fitness functions, multiple seeds
  - Exhaustive search over fine-grained parameter k was impossible

- Multi-Phase Approach
  - Phase A: Preliminary Screening
    - Evaluated all libraries with fewer seeds
    - Explored a wide parameter range using a coarse-grained grid
  - Phase B: Validation
    - Selected promising configurations from Phase A
    - Test with an increased number of seeds
  - Phase C/D: TBD

Analysis on Phase A/B

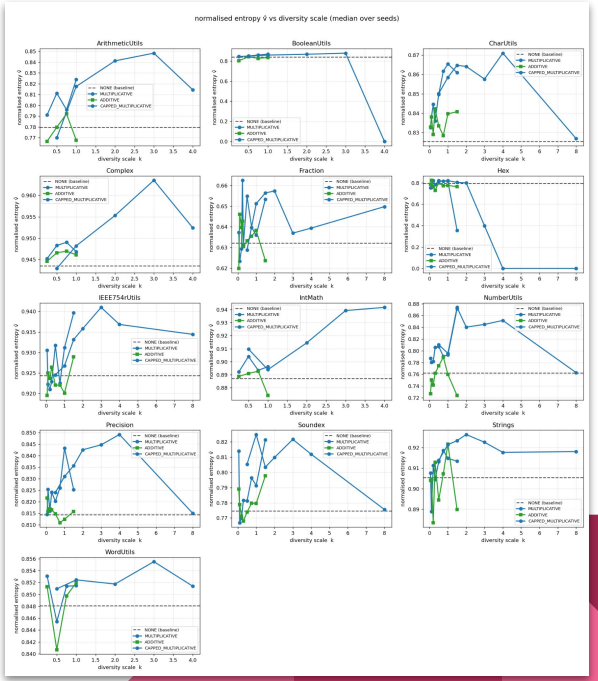
- Plotted results using Python

CS453-team6 / analysis /

Add file

minux-lee chore: move directory — df2a134 · 3 hours ago — History

Name	Last commit message	Last commit date
..		
report	chore: move directory	3 hours ago
__init__.py	chore: move directory	3 hours ago
honest_audit.py	chore: move directory	3 hours ago
metrics.py	chore: move directory	3 hours ago
multi_phase.py	chore: move directory	3 hours ago
parser.py	chore: move directory	3 hours ago
plots.py	chore: move directory	3 hours ago
slide_figures.py	chore: move directory	3 hours ago
slide_fixed_k.py	chore: move directory	3 hours ago



Analysis on Phase A/B: Fitness Functions

- Exponential: Fails to train properly, excluded for phase C/D
- Additive: Stable & promising performance with k in 0-1.5
- Multiplicative:
  - Good performance with k in 0-1.5
  - But completely collapsed when k > 2
  - Hypothesis: Overfitting occurred due to overemphasizing low diversity values

Proposed Solution:

- Adding Capped-Multiplicative mode

Mode	Formula
`NONE`	F_{base}
`MULTIPLICATIVE`	$F_{\text{base}} \cdot (1 + k\delta)$
`ADDITIVE`	$F_{\text{base}} + k\delta$
`EXPONENTIAL`	$F_{\text{base}} \cdot e^{k\delta}$
`CAPPED_MULTIPLICATIVE`	$F_{\text{base}} \cdot (1 + \min(k\delta, c))$

Analysis on Phase A/B: Computing Budget

- Most large classes exhausted the time budget before any evolution
  - Populations were set to 50 by default

- Phase C/D switched to population=5, yielding real evolution
  - ~35 generations on average

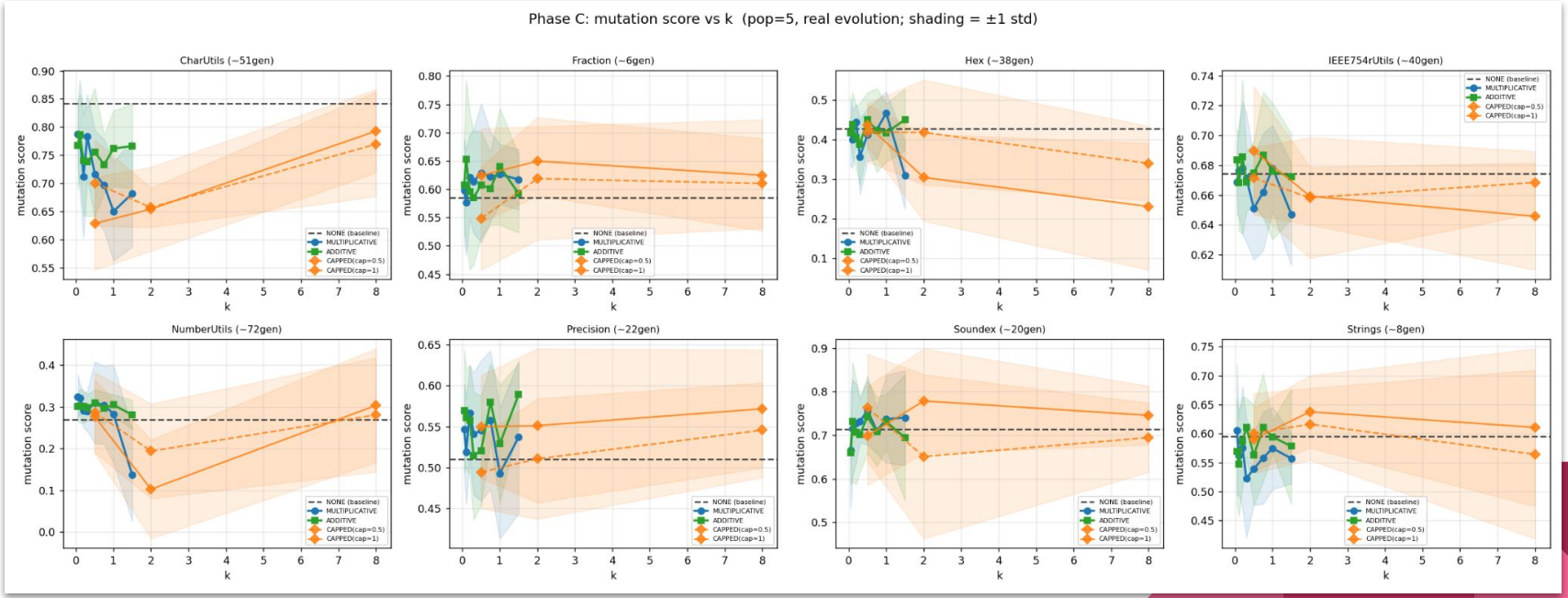
- Larger class needs more budgets, smaller classes needs less.
  - Phase C for small classes
  - Phase D for large classes

Running Phase C/D: Dense Hyperparameter k

Phase	Benchmarks	Coupling	k values	Seeds	Budget	Pop. size
Phase A (explore)	14	4 modes	$\{0.25 \cdots 8\}$ (6)	3	40s	50
Phase B (confirm)	14	Top-5	$\{0.25 \cdots 8\}$	10	40s	50
Phase C (refine)	8	4+CAPPED	$\{0.05 \cdots 1.5\}$ (8)	8	60s	5
Phase D (large)	6	4+CAPPED	$\{0.25 \cdots 4\}$ (5)	6	120s	5
Total						**≈3,790 runs**

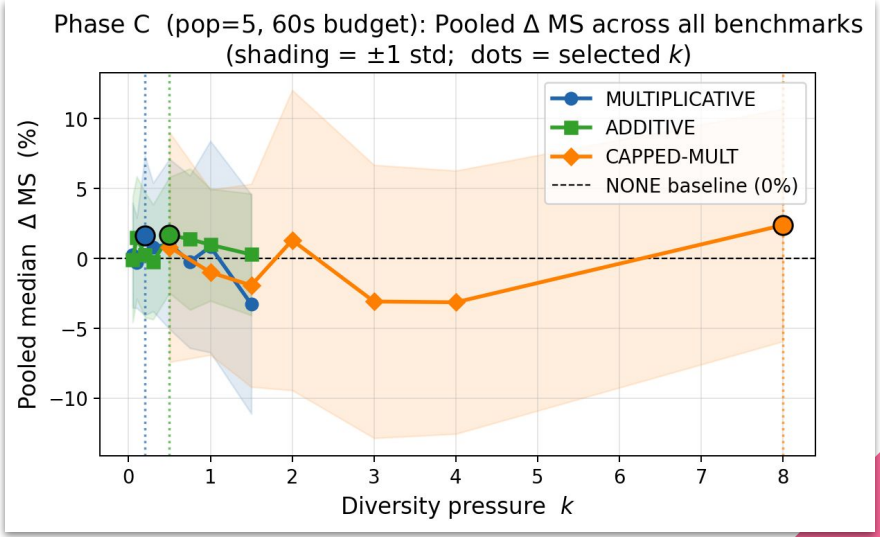
Analysis on Phase C

- Plotted results using Python

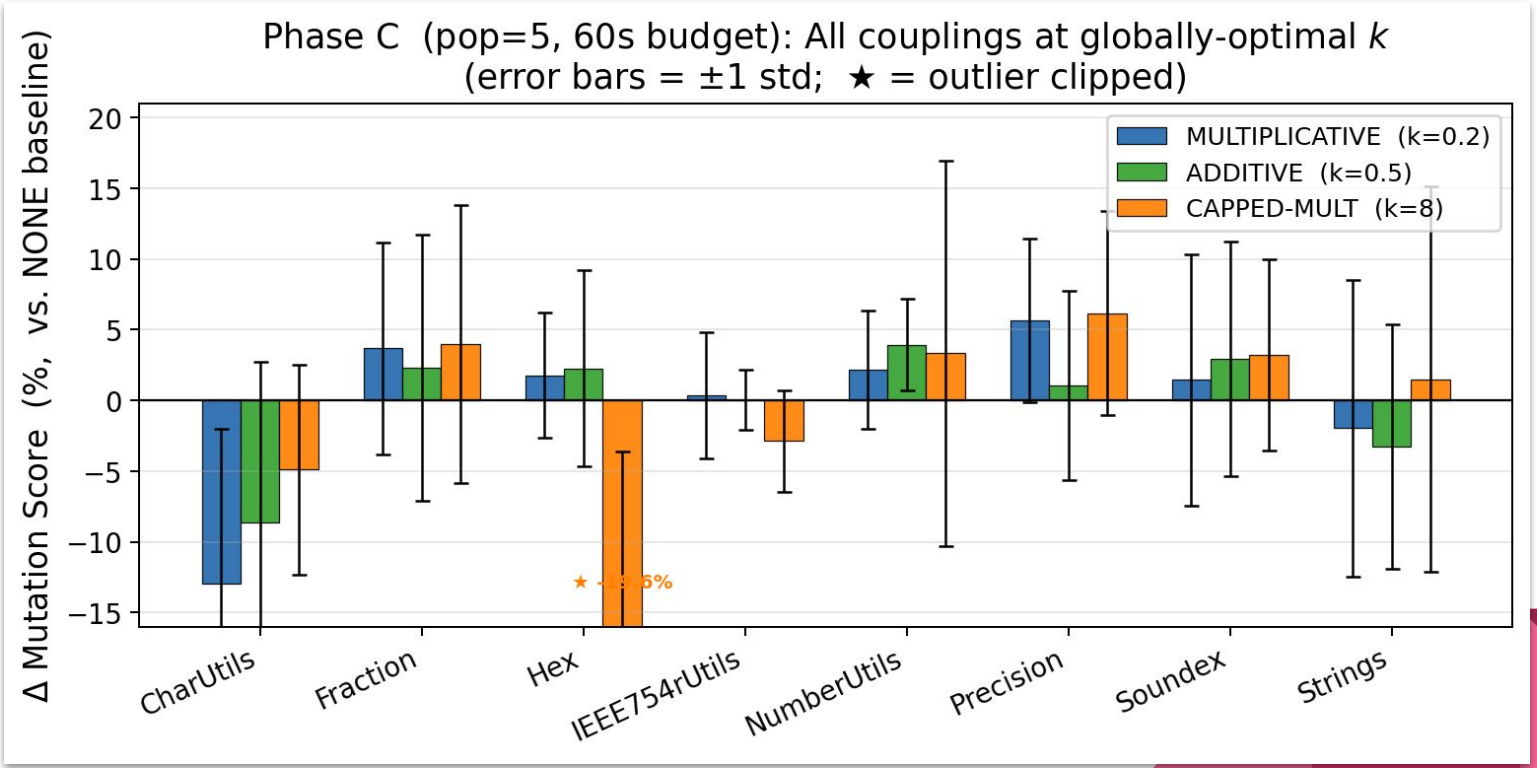


Analysis on Phase C

- Select best-k for each fitness functions, based on average mutation score
  - Additive: k = 0.5
  - Multiplicative: k = 0.2
  - CAPPED-Multiplicative: k = 8

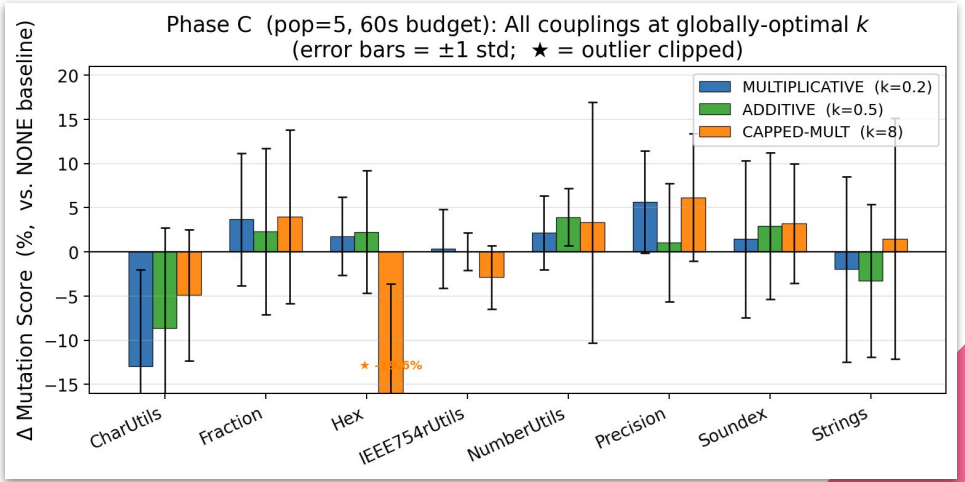


Analysis on Phase C: Mutation Score

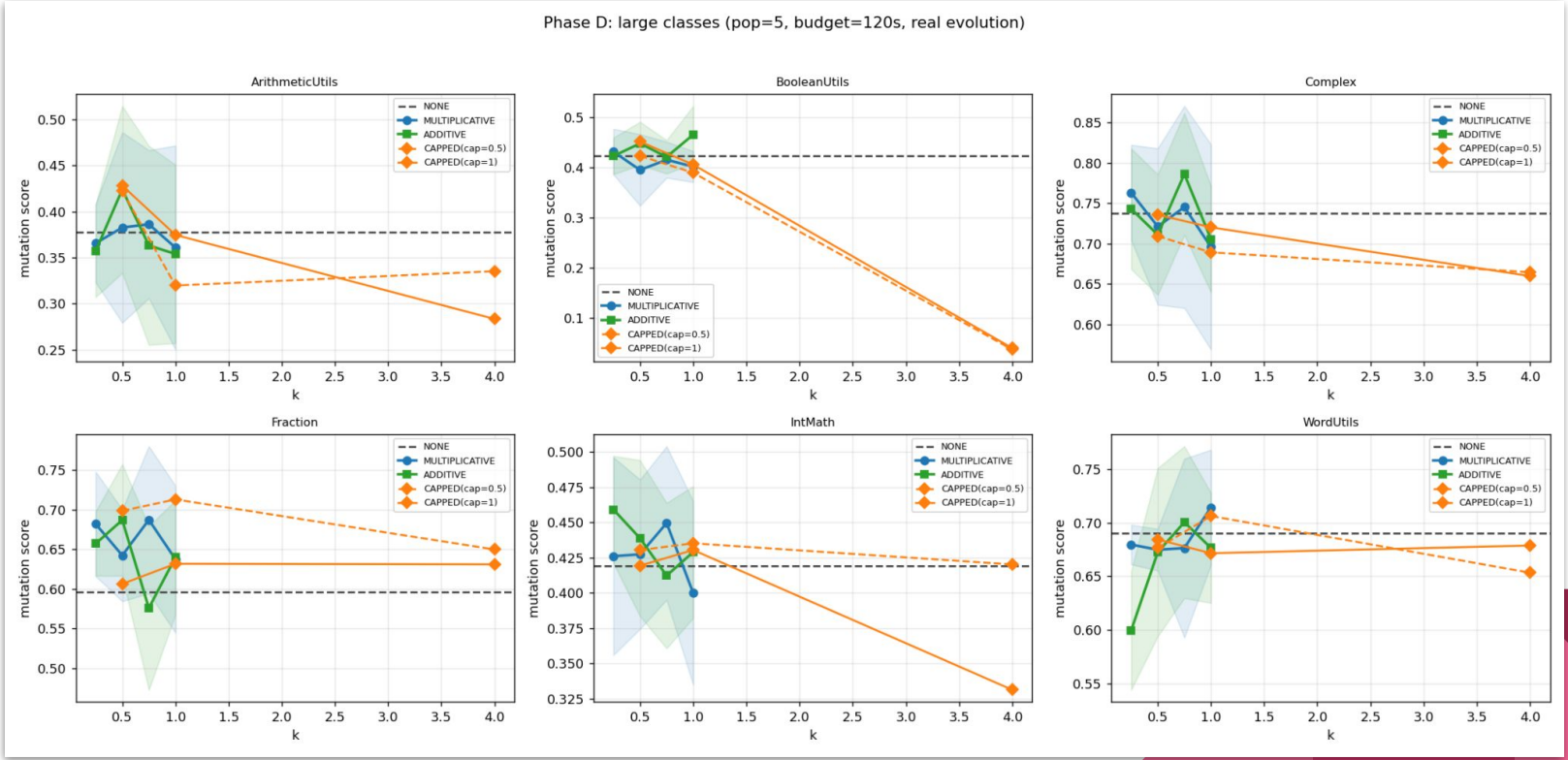


Phase C: CharUtils

- Exceptionally bad result on CharUtils
- Baseline already yields great mutation score and entropy
  - Trade-off occurred:
    - Entropy-guided fitness might shift selection toward diverse operator types at the cost of fewer killed mutants

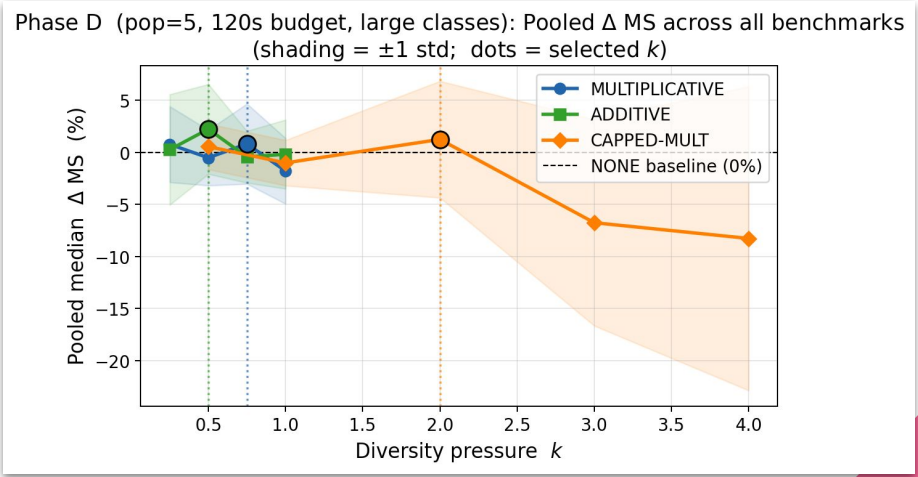


Analysis on Phase D

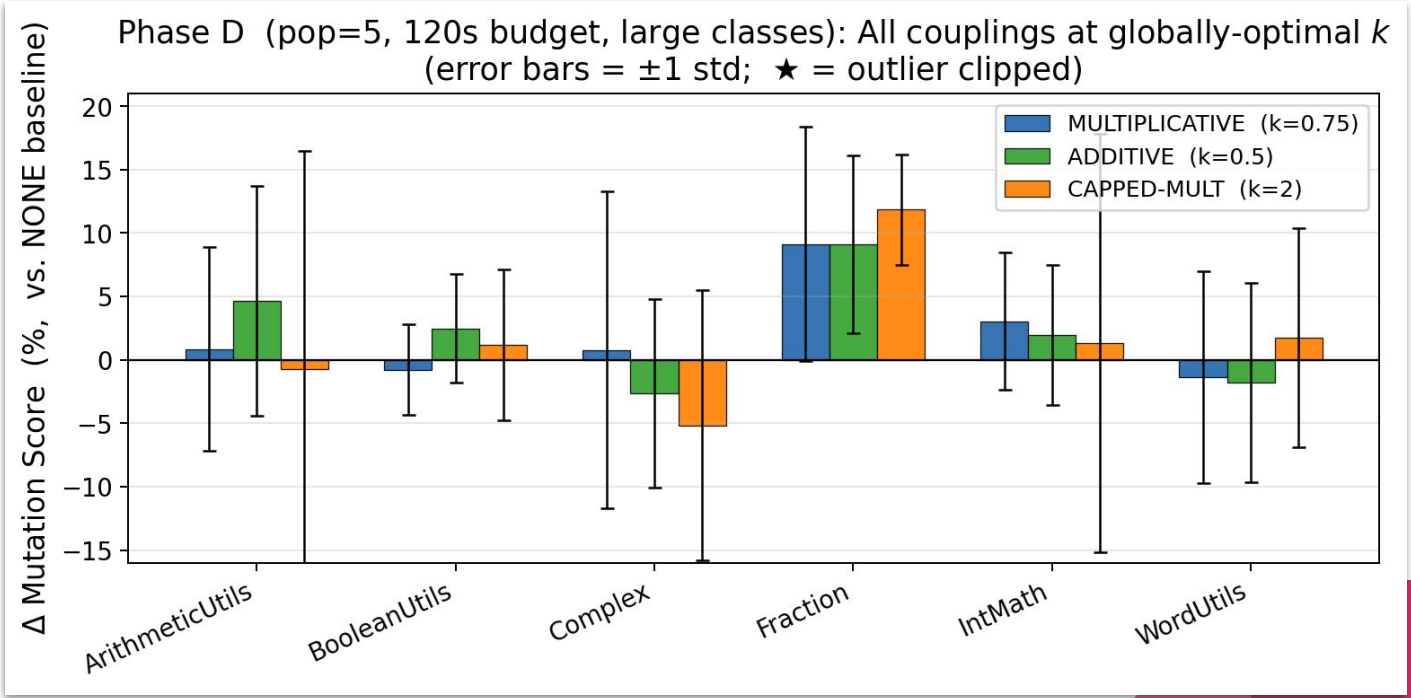


Analysis on Phase D

- Select best-k for each fitness functions, based on average mutation score
  - Additive: k = 0.5
  - Multiplicative: k = 0.75
  - CAPPED-Multiplicative: k = 2

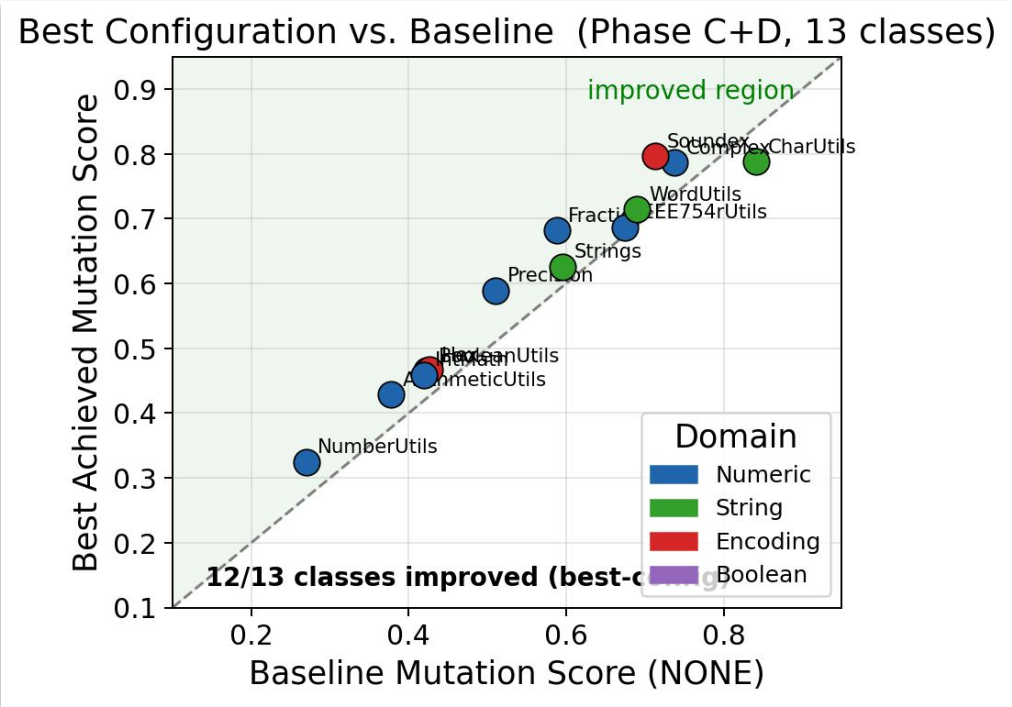


Analysis on Phase D: Mutation score



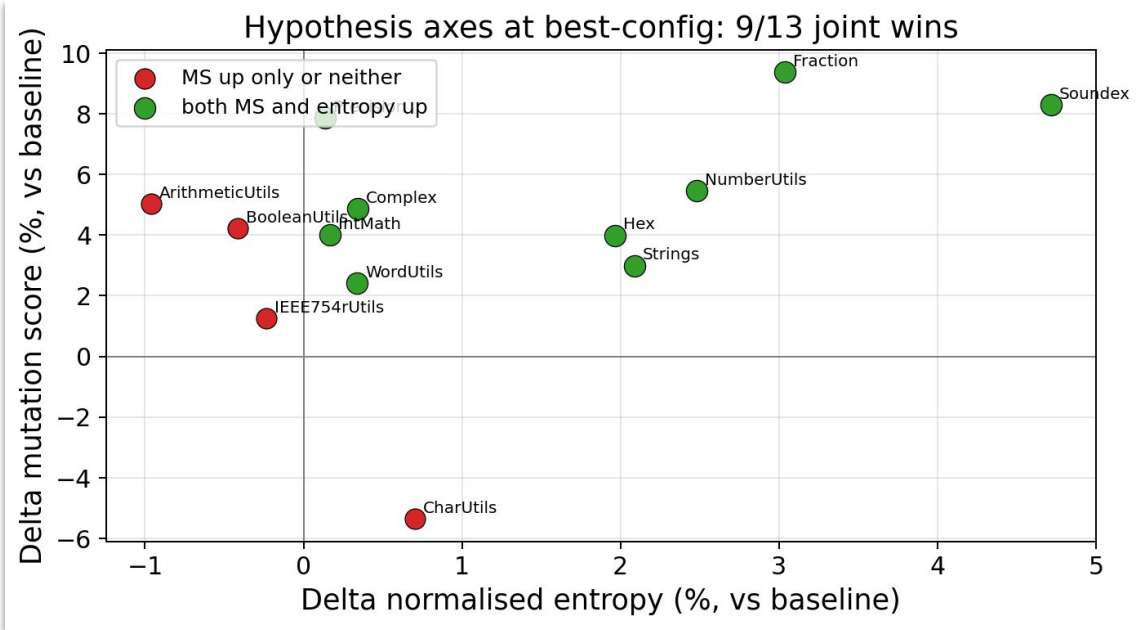
Phase C/D: Summary

- Mutation Score were improved for 12/13 classes under best configuration



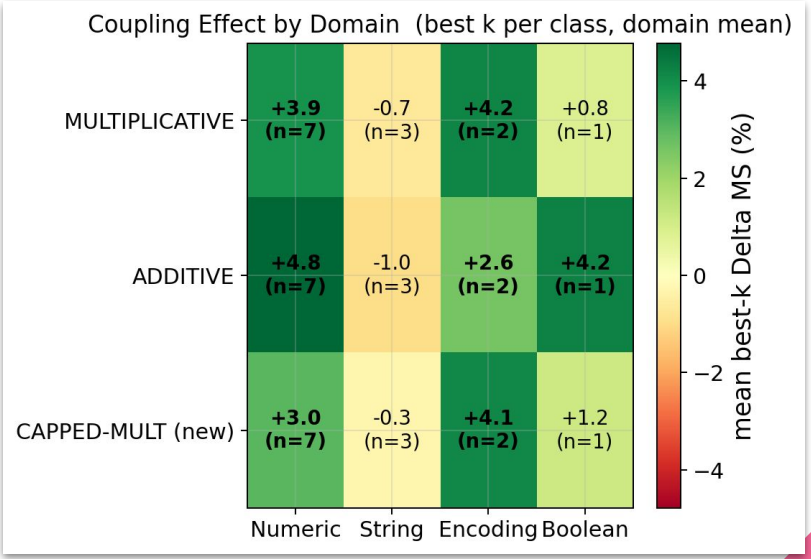
Phase C/D: Correlation between MS and δ

- Both entropy and mutation score were improved for 9/13 classes under best configuration



Phase C/D: Domain Analysis

- Get average Δ mutation score, each for open-source library's domain
- Mutation score was improved in most domains
  - String domain: CharUtils sole exception



Conclusion

- The hypothesis is partially supported:
  - Diversity coupling has potential of improving mutation score on some cases

- Limitation:
  - Diversity-aware fitness is promising but conditional
    - Can’t deploy one formula and one k everywhere
  - Operator entropy is a proxy—real fault detection not tested

- Future work:
  - Test with larger population, more budgets
  - Track each generations to confirm the hypothesis
  - Learn with entropy at early phase, then learn with pure MS at later phase
  - Systematic evaluation of one global setting
  - CAPPED cap selection
  - etc.

References

McMinn, P. (2004). Search-based software test data generation: a survey. STVR, 14(2), 105–156.
Fraser, G., & Arcuri, A. (2011). EvoSuite: automatic test suite generation for Java classes. ISSTA.
Fraser, G., & Arcuri, A. (2014). Achieving higher test quality with mutation-based test generation. TSE, 40(9), 1041–1059.
Just, R., et al. (2014). Are mutants a valid substitute for real faults? FSE.
Papadakis, M., et al. (2019). Mutation testing advances. Advances in Computers, 112, 1–75.

Appendix

GitHub: https://github.com/minux-lee/CS453-team6

Meeting Notes:
https://docs.google.com/document/d/1afdCO0Mh6C1FwdxR_6QQqa1UM66NZW7g3TAgG3Z6Y7E/edit?usp=sharing